AI-First Development

Hands-On Workshop

Build from scratch using AI • 3 Hours • Reference: github.com/emmanuelandre/unveiling-claude

Workshop Agenda (3 Hours)

- Part 1: Foundation (20 min) - Philosophy & Setup
- Part 2: Project Setup (30 min) - Create Your Project
- ■ Break 1 (5 min)
- Part 3: Core Workflow (60 min) - Build a Feature End-to-End
- ■ Break 2 (10 min)
- Part 4: Testing Deep Dive (25 min) - E2E & Unit Tests
- Part 5: Git & Best Practices (20 min) - Commits, PRs, Prompts
- Part 6: Final Challenge (25 min) - Complete Feature Solo
- Wrap-up (5 min)

Part 1: Foundation

The AI-First Philosophy

- Traditional: Human writes code → AI assists
- AI-First: AI executes 100% → Human validates 100%

This means:

- AI handles ALL coding, testing, documentation
- Human handles ALL validation, review, decisions
- Clear handoff points at each step

The 10-Step Development Process

- 1. Specification (Human writes detailed spec)
- 2. Database Schema (AI designs → Human reviews)
- 3. Repository Layer (AI implements → Human reviews)
- 4. API Endpoints (AI creates → Human reviews)
- 5. API E2E Tests (AI writes → Human verifies)
- 6. Frontend Components (AI builds → Human reviews)
- 7. UI E2E Tests (AI creates → Human verifies)
- 8. Documentation (AI updates → Human reviews)
- 9. Code Review (Human conducts)
- 10. Deployment (AI executes → Human verifies)

Workshop Approach

What You'll Build:

- Your OWN project from scratch
- A complete feature with API and tests
- Real Git workflow with commits and PR

Reference Material:

- github.com/emmanuelandre/unveiling-claude
- Contains working examples to study
- Use as patterns, not copy-paste

Choose Your Stack:

- Go, Node, Python - whatever you prefer
- Prompts are stack-agnostic

Prerequisites Check

- ✓ Git 2.x or higher installed
- ✓ Code editor (VS Code, Cursor, etc.)
- ✓ AI coding assistant access (Claude Code, Windsurf, etc.)
- ✓ GitHub account
- ✓ Your preferred language runtime (Go, Node, Python)
- ✓ GitHub CLI (gh) - optional but recommended

Part 2: Project Setup

■ Hands-On: Exercise 1: Create Your Repository

YOUR PROMPT:

```
Create a new GitHub repository for your project:

1. Go to github.com and create a new PRIVATE repository
Name: my-ai-project (or your preferred name)

2. Clone it locally:
git clone https://github.com/[YOUR-USERNAME]/my-ai-project.git
cd my-ai-project

3. Verify:
git status
Should show: "On branch main, nothing to commit"

Reference: Check unveiling-claude repo structure for ideas
```

What You Should See:

- Local repository initialized and connected to GitHub

■ Hands-On: Exercise 2: Create Project Rules File

YOUR PROMPT:

```
Ask your AI assistant:

Help me create a project rules file for my project.

Project details:
- Language: [Go/Node/Python]
- Type: REST API with database
- Database: PostgreSQL (or SQLite for simplicity)
- Testing: E2E with [Cypress/Go test/pytest]

Include these sections:
1. Project Overview (what this project does)
2. Tech Stack (language, framework, database)
3. Project Structure (recommended directories)
4. Commands (build, test, run)
5. Git Workflow (branch naming, commit format)
6. Testing Requirements (E2E mandatory)
```

```
Save as CLAUDE.md (or .windsurfrules for Windsurf)
```

What You Should See:

• Project rules file created with all sections

What Goes in a Project Rules File

Project Overview:

- Brief description of what this project does

Tech Stack:

- Language, frameworks, database, testing tools

Commands:

- How to build, test, run, and lint

Git Workflow:

- Branch naming conventions
- Commit message format

Testing Requirements:

- Coverage expectations
- What must be tested

Hands-On: Exercise 3: Initialize Git Workflow

YOUR PROMPT:

```
Ask your AI assistant:

Set up the git workflow for my project:

1. Create .gitignore for [Go/Node/Python] project
Include: build outputs, dependencies, IDE files, .env

2. Create initial directory structure:
- cmd/ or src/ for source code
- tests/ or test/ for tests
- migrations/ for database migrations
- docs/ for documentation

3. Create initial commit:
git add .
git commit -m "chore: initial project setup"

4. Create feature branch for our first feature:
```

```
git checkout -b feature/user-auth
```

What You Should See:

• Git initialized with proper structure and on feature branch

5-Minute Break

Part 3: Core Workflow - Build a Feature

Feature: User Authentication

We'll build:

- User registration endpoint
- User login endpoint
- JWT token authentication
- E2E tests for all endpoints

Following the 10-step process:

- Step 1: Write specification
- Step 2: Create database schema
- Step 3-4: Implement repository and API
- Step 5: Write E2E tests

• Each step: AI implements → You review → Approve or request changes

■ Hands-On: Step 1: Write Feature Specification

YOUR PROMPT:

```
Ask your AI assistant:

I need to implement user authentication for my API.

Requirements:
- Users register with email and password
- Users login with email and password
- JWT tokens for authentication
- Password hashing (never store plain text)

Database Schema Needed:
- users table: id, email, password_hash, created_at, updated_at

API Endpoints:
POST /api/auth/register - Register new user
Request: { email, password }
Response: { user_id, email, token }
```

```
POST /api/auth/login - Login existing user
Request: { email, password }
Response: { user_id, email, token }

Success Criteria:
- Passwords hashed with bcrypt
- JWT expires after 1 hour
- Proper HTTP status codes (201, 200, 400, 401)
- E2E tests cover happy path and errors

Please confirm you understand before we proceed.
```

What You Should See:

• AI confirms understanding, may ask clarifying questions

Reviewing the Specification

Check that AI understood:

- ■ All requirements captured?
- ■ Endpoints match your expectations?
- ■ Any clarifying questions to answer?

Common clarifications:

- ■ Password minimum length?
- ■ Token expiration time?
- ■ Error message format?

- If satisfied: "Looks good, please create the database migration"
- If changes needed: "Change X to Y because..."

Hands-On: Step 2: Create Database Schema

YOUR PROMPT:

```
Ask your AI assistant:

Create the database migration for the users table.

Requirements:
- Table name: users
- Columns: id (primary key), email (unique), password_hash, created_at, updated_at
- Add index on email column for fast lookups

For Go: Create migrations/001_create_users.up.sql and .down.sql
For Node: Create migrations/001_create_users.js
For Python: Create migrations/001_create_users.py

Use appropriate types for your database:
- PostgreSQL: SERIAL, VARCHAR, TIMESTAMP
- SQLite: INTEGER PRIMARY KEY, TEXT

Include both up (create) and down (drop) migrations.
```

What You Should See:

- Migration file(s) created with proper schema

Expected: Database Migration (SQL)

SQL

```sql
-- migrations/001_create_users.up.sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);

-- migrations/001_create_users.down.sql
DROP TABLE IF EXISTS users;
```

Review Checklist: Database Schema

Check before approving:

- ■ Column types appropriate for your database?
- ■ Primary key defined correctly?
- ■ Unique constraint on email?
- ■ Index on frequently queried columns (email)?
- ■ Timestamps with default values?
- ■ Down migration is safe (doesn't lose other data)?

- If good: "Schema looks good, please implement the repository layer"
- If issues: "Change X to Y..."

■ Hands-On: Step 3: Implement Repository Layer

YOUR PROMPT:

```
Ask your AI assistant:

Implement the repository layer for user operations.

Create a UserRepository with these methods:
- Create(email, passwordHash) -> User
- FindByEmail(email) -> User or null
- FindByID(id) -> User or null

Requirements:
- Use prepared statements (prevent SQL injection)
- Handle database errors gracefully
- Return appropriate errors (not found, duplicate, etc.)

Place in:
- Go: internal/repository/user_repository.go
- Node: src/repositories/userRepository.js
- Python: app/repositories/user_repository.py
```

```
Follow patterns from the project rules file.
```

What You Should See:

• Repository file created with all methods

■ Hands-On: Step 4: Implement API Handlers

YOUR PROMPT:

```
Ask your AI assistant:

Implement the API handlers for authentication.

Create handlers for:
1. POST /api/auth/register
- Validate email format
- Validate password (minimum 8 characters)
- Hash password with bcrypt
- Create user in database
- Generate and return JWT token
- Return 400 for validation errors
- Return 409 for duplicate email

2. POST /api/auth/login
- Find user by email
- Verify password against hash
- Generate and return JWT token
```

```
- Return 401 for invalid credentials

Include:
- Input validation
- Proper HTTP status codes
- JSON response format: { success: true/false, data/error }
- Wire up routes to main application
```

What You Should See:

• Handler files created and routes configured

Expected: API Handler Structure

GO

```
// Pseudo-code structure (adapt for your language)

func Register(request) response {
    // 1. Parse and validate input
    email, password := parseRequest(request)
    if !validEmail(email) {
        return error(400, "Invalid email")
    }
    if len(password) < 8 {
        return error(400, "Password too short")
    }

    // 2. Hash password
    hash := bcrypt.Hash(password)

    // 3. Create user
    user, err := repo.Create(email, hash)
    if err == DuplicateEmail {
        return error(409, "Email already exists")
    }
```

```
    // 4. Generate token
    token := jwt.Generate(user.ID)

    return success(201, { user, token })
}
```

Review Checklist: API Implementation

Security:

- ■ Password hashed before storing?
- ■ No plain text passwords in logs?
- ■ SQL injection prevented (prepared statements)?

Error Handling:

- ■ Appropriate status codes?
- ■ Clear error messages (no internal details exposed)?
- ■ All error paths handled?

Validation:

- ■ Email format validated?
- ■ Password requirements checked?

- Required fields validated?

Hands-On: Step 5: Write E2E API Tests

YOUR PROMPT:

```
Ask your AI assistant:

Create E2E tests for the authentication endpoints.

Test cases needed:
1. Register - Happy path (201)
- New user can register
- Response includes token

2. Register - Duplicate email (409)
- Cannot register same email twice

3. Register - Invalid email (400)
- Rejects malformed email

4. Register - Weak password (400)
- Rejects password < 8 chars
```

```
5. Login - Valid credentials (200)
- Returns token

6. Login - Invalid password (401)
- Wrong password rejected

7. Login - Non-existent user (401)
- Unknown email rejected

Use your testing framework:
- Go: internal/handlers/auth_test.go
- Node: tests/api/auth.test.js (Jest/Vitest)
- Python: tests/test_auth.py (pytest)

Include setup and teardown for test database.
```

What You Should See:

• Test file created with all test cases

Expected: E2E Test Structure

JAVASCRIPT

```
// Example test structure (adapt for your framework)

describe('Auth API', () => {
  beforeEach(() => {
    // Clear test database
  });

  describe('POST /api/auth/register', () => {
    it('registers new user successfully', async () => {
      const response = await request(app)
        .post('/api/auth/register')
        .send({ email: 'test@example.com', password: 'SecurePass123' });

      expect(response.status).toBe(201);
      expect(response.body.token).toBeDefined();
    });

    it('rejects duplicate email', async () => {
      // First registration
      await request(app)
        .post('/api/auth/register')
```

```
      .send({ email: 'test@example.com', password: 'SecurePass123' });

    // Duplicate attempt
    const response = await request(app)
      .post('/api/auth/register')
      .send({ email: 'test@example.com', password: 'SecurePass123' });

    expect(response.status).toBe(409);
  });
 });
});
```

Hands-On: Run and Verify Tests

YOUR PROMPT:

```
Run your E2E tests:

For Go:
go test -v ./internal/handlers/...

For Node:
npm test -- --grep "Auth API"

For Python:
pytest tests/test_auth.py -v

Expected: All 7 test cases pass (green)

If tests fail, share the error with your AI assistant:
"Test [name] is failing with this error:
[paste error output]

Please analyze the failure and suggest a fix."
```

```
Continue until ALL tests pass!
```

What You Should See:

- All E2E tests passing (7/7 green)

10-Minute Break

Part 4: Testing Deep Dive

Testing Philosophy

• Both E2E and Unit tests are MANDATORY

E2E Tests Cover:

■ Complete user journeys

■ API endpoint behavior

■ Integration between components

Unit Tests Cover:

■ Business logic and calculations

■ Utility functions

■ Edge cases and error handling

• Takeaway: E2E catches integration issues, Unit catches logic bugs

Hands-On: Exercise: Add Unit Tests

YOUR PROMPT:

```
Ask your AI assistant:

Add unit tests for the authentication logic.

Create unit tests for:
1. Password validation function
- Test: 8+ chars passes
- Test: < 8 chars fails
- Test: Empty string fails

2. Email validation function
- Test: valid@email.com passes
- Test: invalid-email fails
- Test: Empty string fails

3. JWT token generation
- Test: Token contains user ID
- Test: Token has correct expiration
```

```
4. Password hashing
- Test: Same password produces different hashes (salt)
- Test: Verify function works

Place in appropriate test file:
- Go: internal/auth/auth_test.go
- Node: tests/unit/auth.test.js
- Python: tests/unit/test_auth.py
```

What You Should See:

• Unit tests created and passing

Testing Best Practices

Use data-test attributes for UI:

■ data-test="login-button" not button.primary

Test user journeys, not implementation:

■ "User can log in" not "Login function returns token"

Coverage priorities:

■ 1. Happy path - must work

■ 2. Error cases - validation, auth failures

■ 3. Edge cases - empty strings, nulls

• Run tests in pre-commit hooks

Part 5: Git & Best Practices

Git Workflow Standards

Branch Naming:

- feature/user-auth
- fix/login-bug
- refactor/api-cleanup

Commit Format:

- feat(auth): add user registration
- fix(auth): handle duplicate email error
- test(auth): add E2E tests for login

Hard Rules:

- Never commit to main directly
- Never skip pre-commit checks

- Never commit secrets

Hands-On: Exercise: Create Proper Commits

YOUR PROMPT:

```
Ask your AI assistant:

Help me commit my authentication feature properly.

My changes include:
- Database migration for users table
- Repository layer
- API handlers
- E2E tests
- Unit tests

Steps:
1. Review what's changed: git status
2. Stage all changes: git add .
3. Create commit with conventional format

Suggested commit message:
feat(auth): add user registration and login
```

```
- Add users table migration
- Implement user repository with CRUD
- Create register and login endpoints
- Add E2E tests for all auth endpoints
- Add unit tests for validation logic

After committing, verify with: git log --oneline -1
```

What You Should See:

• Commit created with proper conventional commit message

Hands-On: Exercise: Create Pull Request

YOUR PROMPT:

```
Ask your AI assistant:

Help me push my branch and create a pull request.

Steps:
1. Push branch to remote:
git push -u origin feature/user-auth

2. Create PR (with GitHub CLI):
gh pr create --title "feat: Add user authentication" \
--body "## Summary
- User registration endpoint
- User login endpoint
- JWT token authentication
- E2E tests (7 passing)
- Unit tests for validation

## Testing
```

```
- All E2E tests pass
- All unit tests pass
- Manual testing completed

## Checklist
- [x] Code follows project conventions
- [x] Tests added
- [x] Documentation updated"

Or create PR through GitHub web interface.
```

What You Should See:

• PR created on GitHub with description

Effective Prompt Engineering

Be Specific:

- Not "add validation" but "validate email format and password min 8 chars"

Provide Context:

- Reference existing patterns in the codebase
- Specify where files should be placed

Include Success Criteria:

- "Tests should pass"
- "Return proper HTTP status codes"

Break Down Complex Tasks:

- Number your steps
- Validate after each step

Good vs Bad Prompts

■ Bad

• "Add search"

• "Fix the bug"

• "Make it faster"

• "Add validation"

■ Good

• "Add search: filter by email, case-insensitive, return paginated"

• "Fix: login returns 500 when email contains + character"

• "Add pagination: 20 per page, cursor-based, sort by created_at"

• "Validate: email format, password 8+ chars, both required"

Part 6: Final Challenge

Final Exercise: Complete Feature

Build ONE of these features end-to-end:

- Option A: GET /api/auth/me - Get current user profile
- Option B: PUT /api/auth/password - Change password
- Option C: POST /api/auth/logout - Logout (invalidate token)

Follow the full workflow:

- 1. Write specification
- 2. Update database if needed
- 3. Implement repository and handler
- 4. Write E2E tests
- 5. Add unit tests if applicable
- 6. Commit and add to PR

- Time: 25 minutes

■ Hands-On: Final Challenge: Your Feature

YOUR PROMPT:

```
Choose ONE feature and implement it fully:

OPTION A: GET /api/auth/me
- Returns current user info from JWT token
- Response: { id, email, created_at }
- Tests: valid token returns user, invalid token returns 401

OPTION B: PUT /api/auth/password
- Request: { current_password, new_password }
- Verify current password, update to new
- Tests: success, wrong current password, weak new password

OPTION C: POST /api/auth/logout
- Invalidate current token (add to blacklist)
- Tests: logout succeeds, token no longer works

Steps:
1. Tell AI which feature you chose
```

```
2. Have AI implement it
3. Review the code
4. Run tests
5. Commit with conventional format

Time limit: 25 minutes
```

What You Should See:

• New feature implemented with passing tests and committed

Wrap-Up

What We Built Today

- ✓ Project from scratch with proper structure
- ✓ Project rules file (CLAUDE.md)
- ✓ User authentication feature
- ✓ Database migration
- ✓ Repository and API layers
- ✓ E2E tests (7+ test cases)
- ✓ Unit tests
- ✓ Proper Git workflow
- ✓ Pull request ready for review

Key Takeaways

- AI executes 100%, Human validates 100%
- Always write specs BEFORE AI implements
- Review every piece of AI-generated code
- Tests are mandatory (E2E + Unit)
- Use project rules file to maintain consistency
- Conventional commits and proper Git workflow
- Break complex tasks into smaller steps

Resources & Next Steps

Reference Repository:

- github.com/emmanuelandre/unveiling-claude
- Study the patterns, adapt for your projects

AI Tools:

- Claude Code: claude.ai/code
- Windsurf: codeium.com/windsurf
- Cursor: cursor.sh

Your Next Project:

- Start with a project rules file
- Use the 10-step workflow
- Build good habits from day one

Thank You!

Happy Building! ■